



TERCERA PRÁCTICA

Programación

Curso 2008-2009

**Grado en Ingeniería Informática, Grado en Ingeniería de
Sistemas de Comunicaciones y doble Grado en Ingeniería
Informática y en Administración de Empresas**

Universidad Carlos III de Madrid

Instrucciones generales

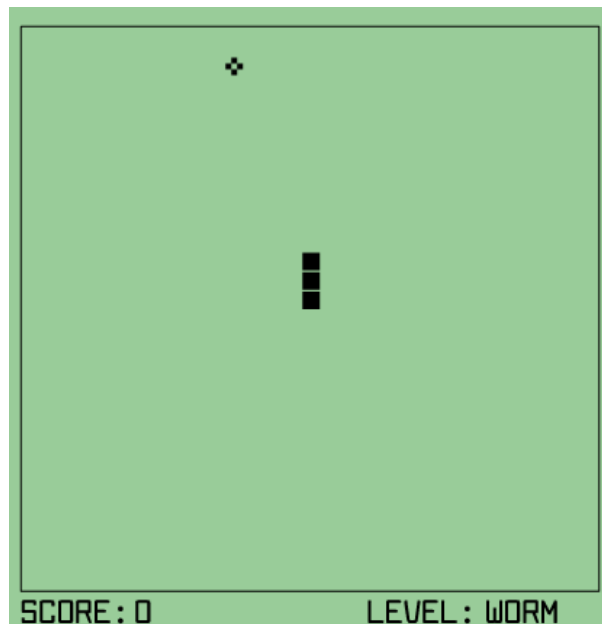
- Durante este curso se deberán realizar **tres prácticas**, cuyas fechas de entrega se pueden encontrar en la página Web de la asignatura:
(<http://galahad.plg.inf.uc3m.es/~docweb/pr-grado/>)
- Las prácticas deberán realizarse **obligatoriamente en grupos de dos alumnos** (hablar con el profesor de prácticas en caso de ser impares).
- La primera práctica se realizará utilizando el programa **Alice**, que puede descargarse gratuitamente de su página web (<http://www.alice.org>) o desde la página web de la asignatura (sección *Software*).
- La segunda y tercera práctica se realizarán en **Java**, usando el entorno de desarrollo Eclipse 3.3 con el J2SE1.6.
- **Se puede entregar cualquier práctica aunque no se haya entregado la anterior.** Se puede cambiar de compañero de grupo entre una práctica y la siguiente.
- La prácticas se calificarán con las siguientes **puntuaciones**:
 - o Primera práctica (Alice): 0,5 puntos
 - o Segunda práctica (Java): 1 punto
 - o Tercera práctica (Java): 1,5 puntos
- Las prácticas podrán tener una **parte obligatoria** y **partes opcionales** que servirán para **subir nota**. Si un alumno realiza solamente la parte obligatoria podrá obtener la nota máxima, aunque no haya realizado ninguna de las partes opcionales. La nota máxima será en cualquier caso la indicada para cada práctica en el apartado anterior.

Tercera Práctica

1. Introducción

El objetivo de la tercera práctica del curso es que los alumnos desarrollen un programa Java inspirado en el clásico juego de la **serpiente (snake)**. En este videojuego, el jugador controla una serpiente que se desplaza por un laberinto de juego sin que colisione con ningún muro ni con su propio cuerpo. Al moverse por el laberinto, la serpiente va “comiendo” objetos que aparecen en las casillas de juego, incrementando así su contador de puntos y su tamaño. El juego termina cuando la serpiente colisiona contra una pared o con una parte de su cuerpo (juego perdido).

En Internet se pueden encontrar numerosas versiones de este juego, como por ejemplo la que existe en <http://www.snakegame.net/phonesnake.htm> cuya pantalla se incluye abajo. En el caso mostrado las únicas paredes son las exteriores, pero en un caso genérico podría haber también paredes interiores. El alumno puede probar alguna de estas versiones para entender mejor el desarrollo del juego y lo que se pretende realizar en las prácticas de este curso.



En el juego que se va a desarrollar, la acción transcurre en un laberinto con muros interiores en el que habrá, al mismo tiempo, más de un objeto de los que la serpiente se alimenta. Estos objetos representan a ratones que pueden desplazarse por el laberinto.

2. Objetivos docentes

Los objetivos docentes de la primera práctica son:

- Hacer que el alumno continúe su familiarización con la sintaxis de Java: tipos de datos, instrucciones, ficheros que genera, etc.
- Capacitar al alumno para la creación de clases Java, con sus correspondientes métodos y atributos, así como para encontrar la solución de problemas utilizando la metodología de orientación a objetos.

3. Trabajo a realizar

- Definir la clase **Posicion** para representar la posición de un objeto en el *Laberinto*:
 - o Deberá tener 2 atributos de tipo entero, denominados **x** e **y** que guardarán las coordenadas de la posición del objeto.
 - o Crear un método **getX** que devuelva el valor de x y otro **getY** que devuelva el de y. Crea un método **setX** que reciba como parámetro un entero y lo use para dar valor al atributo x. Crear otro **setY** para y.
 - o Crear un constructor para crear objetos de esta clase que reciba los valores de x e y, y un constructor por defecto que use el anterior y cree un objeto en la posición (0,0) (esquina superior izquierda de la pantalla)
- Definir la clase **Casilla** que se utilizará para representar cada una de las casillas del *Laberinto*.
 - o Deberá incluir un atributo denominado **tipo** para guardar el tipo de casilla (muro/vacía).
 - o Deberá incluir un atributo **posicion** que contenga un objeto de tipo **posicion**.
 - o Crear un método denominado **getTipo** que no recibirá parámetros y que devolverá el tipo de una casilla.
 - o Crear un método denominado **setTipo** que no devuelva nada, que reciba como parámetro un valor válido para el tipo de una casilla y que cambie el valor del atributo tipo.
 - o Crear un método denominado **obtenerSimbolo** que no recibirá parámetros y que devolverá un char. El char a devolver será “#” si el tipo es muro y “ ” (espacio) si el tipo es vacío.
 - o Crear un constructor para esta clase que reciba el tipo de Casilla y la posicion en la que está.
 - o Crear un constructor por defecto que cree un muro en la posición (0,0)
- Definir la clase **Serpiente** para representar a la serpiente que protagoniza el juego. La clase debe contener los métodos necesarios para desplazar la serpiente por el *laberinto*:
 - o Deberá contener un atributo denominado **cuerpo** compuesto por un array de objetos **Posicion** que almacene todas las casillas que ocupa la serpiente, llamadas secciones de la serpiente, siendo la primera la que indica la posición de la cabeza.
 - o Deberá contener un atributo, denominado **direccion**, para almacenar la dirección actual en la que se mueve la serpiente (hacia arriba, hacia abajo, hacia la derecha o hacia la izquierda)
 - o Deberá contener un atributo denominado **comido** que indique si la serpiente se acaba de comer un ratón.
 - o Crear un método **cambiaDireccion** que reciba como parámetro la nueva dirección de la serpiente y la cambie y otro **getDireccion** que devuelva la dirección de la serpiente.
 - o Crear métodos **getComido** y **setComido** que devuelvan y den valor al atributo **comido**, respectivamente.
 - o Crear un método **getLongitud** que devuelva la longitud de la serpiente.
 - o Crear un método **coincideCabeza** que reciba una posicion en forma (x,y) y devuelva verdadero si esa posición es la de la cabeza de la serpiente.
 - o Crear un método **coincideCuerpo** que reciba una posicion en forma (x,y) y devuelva verdadero si en esa posición está alguna porción del cuerpo de la serpiente.

- o Crear un método denominado **muevete** para mover la serpiente. Si la serpiente no se ha comido ningún ratón la primera sección de la serpiente, correspondiente a la cabeza, avanzará una posición en la dirección actual, la segunda sección pasará a la posición ocupada anteriormente por la cabeza, la tercera pasará a la posición ocupada por la segunda, etc. Si se ha comido un ratón, se hará lo mismo pero aumentando en uno la longitud de la serpiente por la cola, de forma que la posición en la que estaba la última casilla de la cola siga ocupada.
 - o Crear un constructor para crear objetos de esta clase que reciba un array de objetos Posicion y su dirección actual.
 - o Crear otro constructor por defecto que la cree en dirección hacia arriba, con tamaño tres, el cuerpo en vertical y la cabeza en la posición (5,5).
- Definir la clase **Raton** para representar a los ratones que sirven de comida a la serpiente. La clase tendrá, al menos, atributos para contener la siguiente información acerca de un ratón:
 - o Identificador único de ratón.
 - o Puntuación que recibe la serpiente al comérselo.
 - o Número de ratones que se han creado hasta el momento.
 - o Posición en la que se encuentra el ratón.
 - o Crear un constructor que reciba como parámetros un identificador de ratón, la posición y la puntuación.
 - o Crear un constructor que reciba solamente la posición y que cree un ratón con identificador "ratón"+número de ratones creados hasta el momento y puntuación 100.
- Definir la clase **Jugador** que contendrá el nombre y la puntuación de un jugador.
 - o Crear los atributos, constructores y métodos de acceso adecuados.
- Definir la clase **Record** que contendrá un array de 5 objetos de la clase *Jugador* con la información (nombre y puntuación) de los 5 mejores jugadores.
 - o Crear los atributos, constructores y métodos de acceso necesarios.
 - o Definir el método **void nuevoRecord(Jugador j)** para que dado un objeto *j* de la clase *Jugador* lo inserte en la posición correspondiente del array de 5 mejores puntuaciones.
- Definir una clase **Laberinto** para representar el lugar por el que se desplazan la serpiente y los ratones.
 - o La clase deberá contener atributos para almacenar al menos la siguiente información:
 - El alto y ancho del tablero de juego.
 - Un array para representar las casillas que componen el tablero de juego.
 - La serpiente.
 - Un array de objetos ratón.
 - La puntuación actual.
 - La lista de los mejores 5 jugadores.
 - o Crear un constructor por defecto que cree el siguiente tablero de juego de 29x31 ("@" identifica a la cabeza de la serpiente, "O" al resto de secciones de serpiente, "*" a los ratones, las paredes vacías se representan mediante espacios y "#" son los muros):

```
#####
#                                           #
# *                                           * #
#                                           #
# #####                                     ##### #
# #                                           # #
# #                                           # #
# #                                           # #
# #                                           # #
# #                                           # #
# #                                           # #
# #                                           # #
# #                                           # #
# #                                           # #
#                                           # #
#                                           # #
# @                                           #
# 0                                           #
# 0                                           #
#                                           #
# #                                           # #
# #                                           # #
# #                                           # #
# #                                           # #
# #                                           # #
# #                                           # #
# #                                           # #
# #                                           # #
# #                                           # #
# #####                                     ##### #
#                                           #
# *                                           * #
#                                           #
#####
```

- o Crear un constructor que reciba valores para todos los atributos de la clase Laberinto
- o Crear un método para detectar si la serpiente se ha comido un ratón, en cuyo caso cambiará el atributo adecuado de la clase serpiente y aumentará la puntuación
- o Crear un método para generar un ratón en una posición aleatoria vacía (no ocupada por un muro, por la serpiente o por otro ratón) del laberinto, debe recibir como parámetro la posición del array de ratones en la que se debe crear el ratón.
- o Crear un método para comprobar si se ha perdido la partida (la serpiente ha chocado contra un muro o sí misma). Si se ha perdido, se mostrará el mensaje "GAME OVER" por pantalla, pedirá el nombre al jugador en caso de que haya conseguido una de las 5 mejores puntuaciones, mostrará una tabla con las 5 mejores puntuaciones.
- o Crear un método que devuelva un String con la representación del estado actual del tablero.
- o Crear un método que reciba un dato por teclado, que podrá ser uno de las siguientes:
 - O: para que la serpiente gire 90° a su izquierda.
 - P: para que la serpiente gire 90° a su derecha.
 - Q: para salir del juego.
 - Cualquier otra tecla no modificará la dirección de la serpiente, pero la moverá en la dirección actual, si fuese posible.
- o Crear un método para actualizar el laberinto que mueva la serpiente una casilla en la dirección adecuada, compruebe si se ha comido un ratón o ha chocado contra un muro y actúe en consecuencia, imprimiendo el laberinto al final.

- Realizar un programa de prueba en una clase **Practica3**, en cuyo método main se cree un tablero de juego y se permita jugar al juego de la serpiente.

4. Actividades optativas

- Modificar la “**inteligencia**” de los ratones para que huyan de la serpiente, creando comportamientos diferentes para cada uno de ellos.
- **Variar la puntuación** otorgada por comer a cada ratón con el paso del tiempo. Esta puntuación valdrá 100 al crear al ratón e irá disminuyendo en una unidad cada vez que el usuario pulse una tecla.
- Construir un sistema mediante el cual el jugador pueda jugar **varias pantallas**. Cada pantalla se completará al cumplir una serie de objetivos (comer 20 ratones, por ejemplo) y tendrá un laberinto diferente. El sistema inicializará todos sus valores, como al comienzo del juego, a excepción de la puntuación del protagonista.
- **Almacenar en un fichero** las puntuaciones de los jugadores y cargarlas al principio de cada partida (se proporcionará código de ayuda a los alumnos interesados).

5. Normas de entrega

Se deberá **subir a aula global el código fuente** de la práctica realizada, hasta las 24:00 horas del día 14 de Diciembre de 2008. Se deberá subir un fichero .zip con nombre p3xxxyyy.zip (donde xxx e yyy son las iniciales de los componentes del grupo de prácticas) que contendrá:

- El código fuente implementado por los alumnos, es decir, **SÓLO** los archivos .java implementados por los alumnos.
- practica3.doc o practica3.pdf: Memoria explicativa de los desarrollos realizados que, en general, no debe contener listados de código, salvo los imprescindibles para la correcta explicación del mismo. La estructura de la memoria será aproximadamente la siguiente:
 - o **Portada**: indicando la titulación, la asignatura, el curso, el número de práctica y, para cada uno de los integrantes de la pareja, su nombre, NIA, grupo y campus.
 - o **Índice**: tabla de contenidos del documento.
 - o **Introducción**: breve presentación del documento y su estructura.
 - o **Manual técnico**: descripción (no el código) de las clases implementadas, indicando sus atributos y métodos, así como la relación existente entre ellas. Se deben justificar las decisiones tomadas.
 - o **Manual de usuario**: descripción del uso del programa realizado.
 - o **Mejoras opcionales**: descripción de las mejoras introducidas sobre la propuesta inicial.
 - o **Conclusiones**: conclusiones extraídas durante la realización de la práctica, comentarios personales, críticas constructivas, problemas encontrados, etc.
- **No se corregirán prácticas que no sigan estos criterios**

6. Criterios de evaluación

La práctica se evaluará de acuerdo a los siguientes criterios:

- **FUNCIONALIDAD**:
 - o Mostrar y actualizar laberinto: 0,2 puntos
 - o Movimiento de la serpiente: 0,2 puntos
 - o Creación de ratones: 0,1 puntos

- o Mostrar mejores puntuaciones: 0,1 puntos
- o Detección de final de partida: 0,1 puntos
- MEMORIA:
 - o Presentación, claridad, **ortografía**: 0,1 puntos
 - o Calidad manuales técnico y usuario: 0,2 puntos
 - o Justificación de las decisiones de diseño tomadas: 0,1 puntos
- CÓDIGO:
 - o Modularidad/encapsulación: 0,1 puntos
 - o Calidad implementación (ahorro computación /control de errores): 0,1 puntos
 - o Claridad del código y calidad de los comentarios: 0,2 puntos
- TEMAS OPCIONALES:
 - o Variar puntuación otorgada por cada ratón: 0,05 puntos
 - o Modificar la inteligencia de los ratones: 0,1 puntos
 - o Sistema de varias pantallas: 0,1 puntos
 - o Almacenar y cargar puntuaciones de fichero: 0,2 puntos
 - o Otras mejoras serán tenidas en cuenta por los profesores de prácticas.

IMPORTANTE: Es requisito fundamental para calificar la práctica que el código entregado compile sin errores con el kit de desarrollo JAVA j2SE 1.6. y que se sigan las instrucciones de entrega indicadas en el apartado anterior.

7. Anexo: Código de Ayuda

El código de ejemplo presentado a continuación puede resultar útil al alumno al realizar algunas de las labores necesarias para la práctica:

1) Obtención de tecla por parte del usuario:

```
import java.io.InputStreamReader;
import java.io.BufferedReader;
import java.io.IOException;

public class EjemploTecla {
    public String leeTeclado(){
        System.out.print("Introduce tu nombre y pulsa enter:");
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
        String leído = null;
        try{
            leído = br.readLine();
        } catch(IOException ioe) {
            System.out.print("Excepción al leer de teclado");
            System.exit(-1);
        }
        return leído;}
}
```

2) Generación de números aleatorios:

```
import java.util.Random;
public class EjemploAleatorio {
    public static void main(String[] args) {
        Random rnd = new Random(System.currentTimeMillis());
        int numero = rnd.nextInt(10);
        System.out.println("Número aleatorio generado (0-9): "+numero);
    }
}
```